Advanced Computer-Aided VLSI System Design

Final Project:
Recommend Develop Steps

lecturer: Yueh-Feng Tsai

Graduate Institute of Electronics Engineering, National Taiwan University



NTU GIEE

Data Preparation

- Decompress the 1132_final.tar.gz file

```
tar -zxvf 1132_final.tar.gz
```

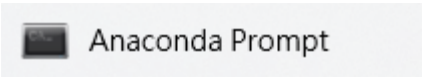
- The data in the tar.gz file

Folder	File	Description
00_TESTBED	TESTBED.sv	Testbench for designs
00_TESTBED	HOST.svp	Host Verilog file
00_TESTBED/PATTERN/IMAGES	image_*.bin	Binary data of images
00_TESTBED/PATTERN	categories.csv	Golden categories
00_TESTBED/PATTERN	confidences.csv	Golden confidences
00_TESTBED/PATTERN	model.dat	Soft link to model.dat
01_RTL	rtl_sim.f	File list for RTL simulation
01_RTL	01_run	RTL simulation script file
01_RTL	resacc.v	Your design

Data Preparation

Folder	File	Description
02_SYN	resacc.sdc	SDC file for synthesis
03_GATE	01_run	Gate simulation script file
03_GATE	gate_sim.f	File list for gate simulation
05_POST	post_sim.f	File list for post layout simulation
05_POST	01_run	Post layout simulation script file
05_POST	02_fsdb2vcd	FDSB to VCD script file
06_POWER	01_power	Power analysis script file
06_POWER	pt.tcl	Primetime script
10_AXI_DRAM	AXI_DRAM.svp	DRAM Verilog file
98_ALGORITHM	environment.yml	Library setting file
98_ALGORITHM	model.dat	Example model.dat
98_ALGORITHM	resnet6.ipynb	Resnet6 Jupyter Notebook
98_ALGORITHM	resnet6.pth	Resnet6 parameter
98_ALGORITHM	resnet6_quantize.ipynb	Quantize Jupyter Notebook

Software Environment Setup

- Install Anaconda
  - Anaconda is a Python environment management tool
  - Download Anaconda Distribution | Anaconda
- Open the anaconda terminal
  - Windows: 

- Go the directory 1132_final/98_ALGORITHM

```
cd [directory]/1132_final/98_ALGORITHM
```

- Setup the environment

```
conda env create -f environment.yml
```

Software Environment Setup

- Activate the environment

```
conda activate 1132-acvsd-final
```

- Change the kernel name

```
python -m ipykernel install --user --name 1132-acvsd-final
--display-name "1132-acvsd-final"
```

- Start the jupyter notebook

```
jupyter notebook
```

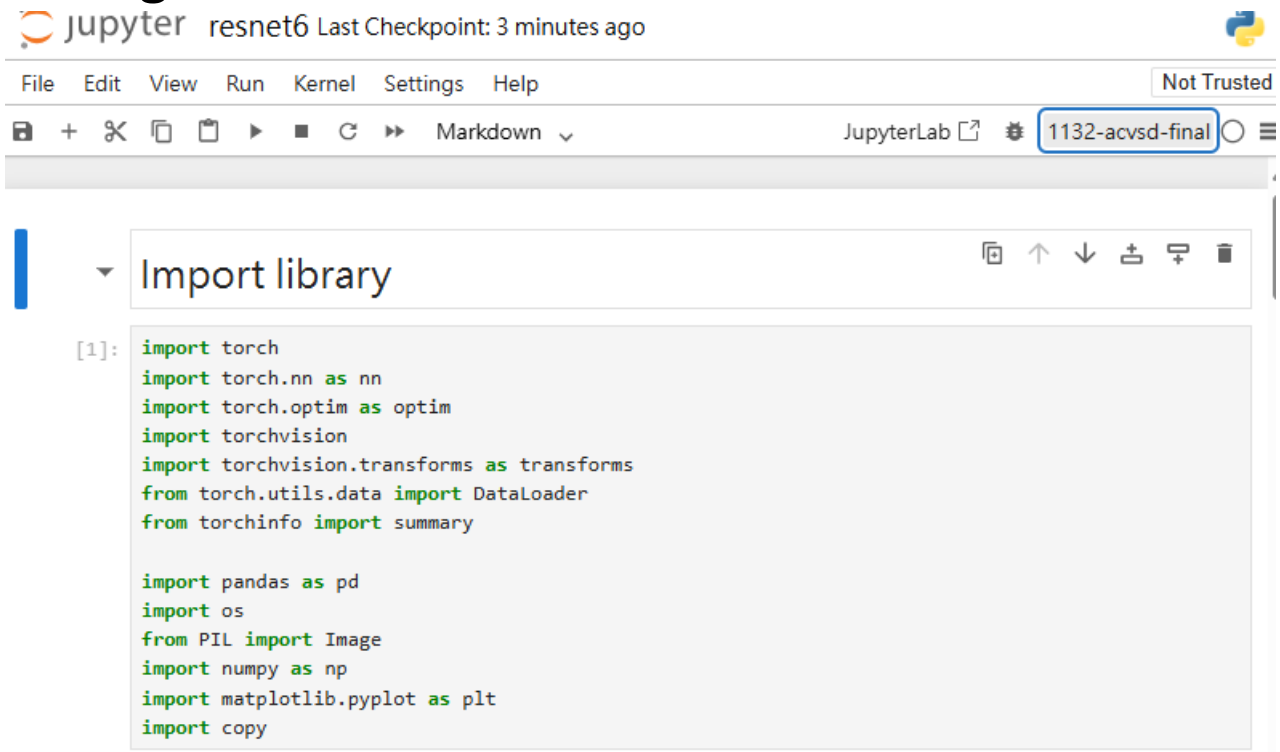
- The file will shown

/

Name	Modified	Size
resnet6_quantize.ipynb	1 hour ago	31.4 KB
resnet6.ipynb	1 minute ago	383.8 KB
enviroment.yml	20 minutes ago	274 B
resnet6.pth	1 hour ago	93.7 KB

Software Environment Setup

- Open resnet6.ipynb or resnet6_quantize.ipynb
- Change the kernel to 1132-acvsd-final



- Check the model configuration and the quantization code

Modify the Parameter and Build model.dat

- Quantize or Encode the parameter of the model
  - Re-train or quantize aware training is prohibited
- Save the model weight and any other information you need to operate the model
  - Use python script to convert (**dat_gen.py** or **dat_gen.ipynb**)
  - [directory]/1132_final/98_ALGORITHM/model.dat

```
1   @10000
2   A3 5F 2D C8
3   9B 71 E6 3C
4   48 F2 07 BE
5   61 8E 94 B0
6   2A 56 19 D4
7   77 3B 01 9F
8   EA 6C C1 12
9   88 0E A7 3F
10  45 D2 9C 34
11  A0 5B 8D 7E
```

RTL Simulation

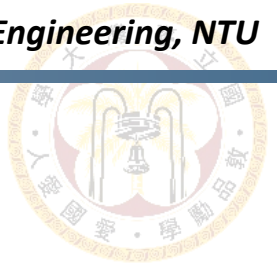
- Design your ResNet accelerator
  - You may set the latency of DRAM to 0 and the clock period the same at the early stage of the design
  - You can also set the batch size to 1 at the early stage
  - Remember to set the latency and period back
- Change the CONFIDENCE_SCALE in rtl_sim.f
  - The confidence value printed is $output/CONDIDENCE_SCALE$

```
##########################################################
# define
##########################################################
+define+TOTAL_IMAGE_NUMBER=64
+define+BATCH_IMAGE_NUMBER=16
+define+CONFIDENCE_SCALE=1048576
+define+RESACC_CLK_PERIOD=1
+define+READ_LATENCY=12
+define+WRITE_LATENCY=16
+define+RTL
```

RTL Simulation

- List all your file to the **`rtl_sim.f`**
- Run **`01_run`**
- Record the error rate and the latency

```
**************************************************************************
                Complete  64 Images!
                Correct Count:   5
                Error Count:  59
                Error Rate: 0.922
                TOTAL LATENCY: 29779483.000 ns
**************************************************************************
$finish called from file "../00_TESTBED/HOST.sv", line 310.
$finish at simulation time          29779555000
           V C S   S i m u l a t i o n   R e p o r t
Time: 29779555000 ps
CPU Time:     45.360 seconds;       Data structure size:   0.0Mb
Sat Apr  5 18:45:15 2025
CPU time: 45.394 seconds in simulation
```



Synthesis

- Modify the clock period constraint of clk_dut **`resacc.sdc`** file
- Create the synthesis tcl file
  - Use the ss0p72vm40c corner for synthesis
- Record the area after synthesis

```
Number of ports:
Number of nets:
Number of cells:
Number of combinational cells:
Number of sequential cells:
Number of macros/black boxes:
Number of buf/inv:
Number of references:

Combinational area:              17263.135097
Buf/Inv area:                     3703.605144
Noncombinational area:            6991.660891
Macro/Black Box area:             4951.919189
Net Interconnect area:      undefined  (Wire load has zero net area)

Total cell area:                 29206.715178
Total area:                 undefined
```

Gate/PR Simulation

- Change the CONFIDENCE_SCALE in *_sim.f
- Run **`01_run`**
  - Please check whether you have successfully include the sdf annotation to your design
- Run **`02_fsdb2vcd`** to create the VCD file for power analysis
- Check the predicted category and confidence values are the same as the values in the RTL simulation

Power Analysis

- Run **`01_power`**
- Record the analysis result

```
                 Internal  Switching  Leakage    Total
Power Group      Power     Power      Power      Power   (     %)  Attrs
--------------------------------------------------------------------------
clock_network       0.0000    0.0000    0.0000    0.0000 ( 0.00%)  i
register            0.0000    0.0000    0.0000    0.0000 ( 0.00%)
combinational    9.278e-05 9.930e-05 5.234e-06 1.973e-04 (11.83%)
sequential       8.143e-04 1.671e-05 4.455e-06 8.355e-04 (50.09%)
memory           6.343e-04 4.868e-09 7.515e-07 6.350e-04 (38.08%)
io_pad              0.0000    0.0000    0.0000    0.0000 ( 0.00%)
black_box           0.0000    0.0000    0.0000    0.0000 ( 0.00%)

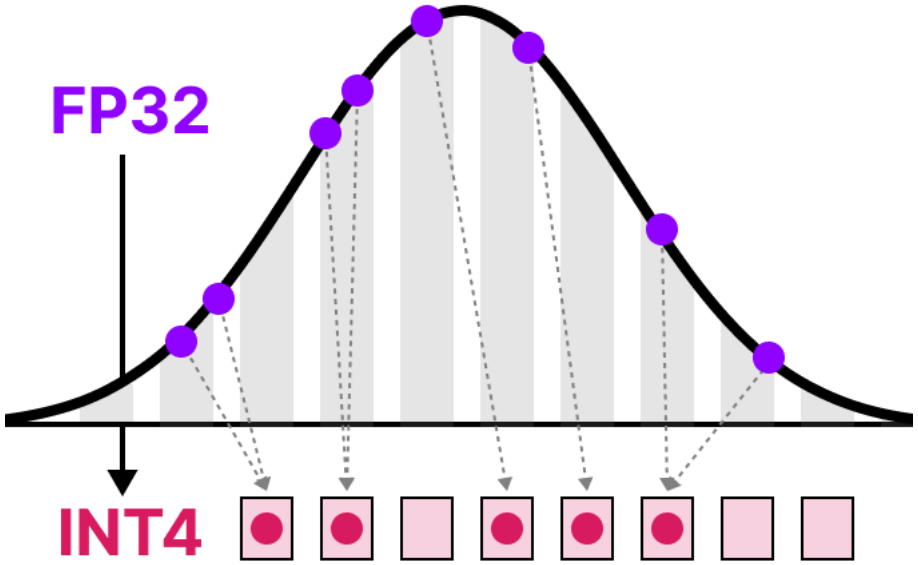
  Net Switching Power  = 1.160e-04   ( 6.96%)
  Cell Internal Power  = 1.541e-03   (92.42%)
  Cell Leakage Power   = 1.044e-05   ( 0.63%)
                         ---------
Total Power            = 1.668e-03  (100.00%)

X Transition Power     = 2.031e-06
Glitching Power        =    0.0000

Peak Power             =    0.6519
Peak Time              =    60.055
```

Model Quantization

- Model quantization can reduce the computing and memory resource requirements
- The number of bits of weight should depend on the dynamic range of the weight and accuracy required

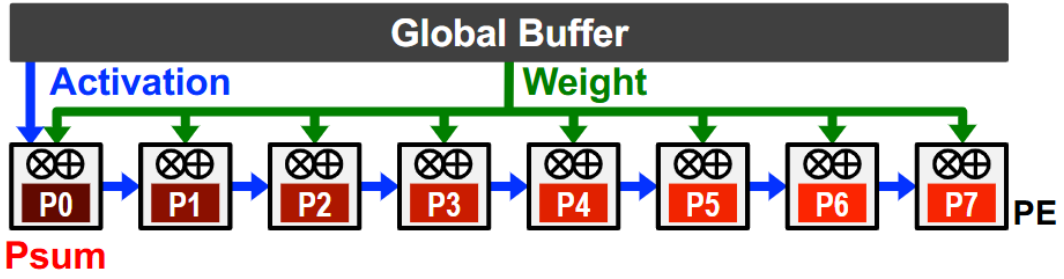


Source: A Visual Guide to Quantization - by Maarten Grootendorst

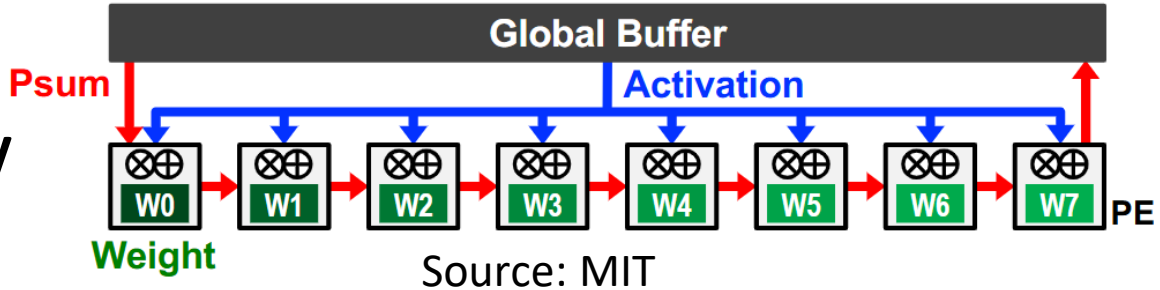
Data Scheduling

- Three primary data scheduling methods
  - Output Stationery
  - Weight Stationery
  - Input Stationery
- Proper data scheduling can reduce external memory access

Output Stationery



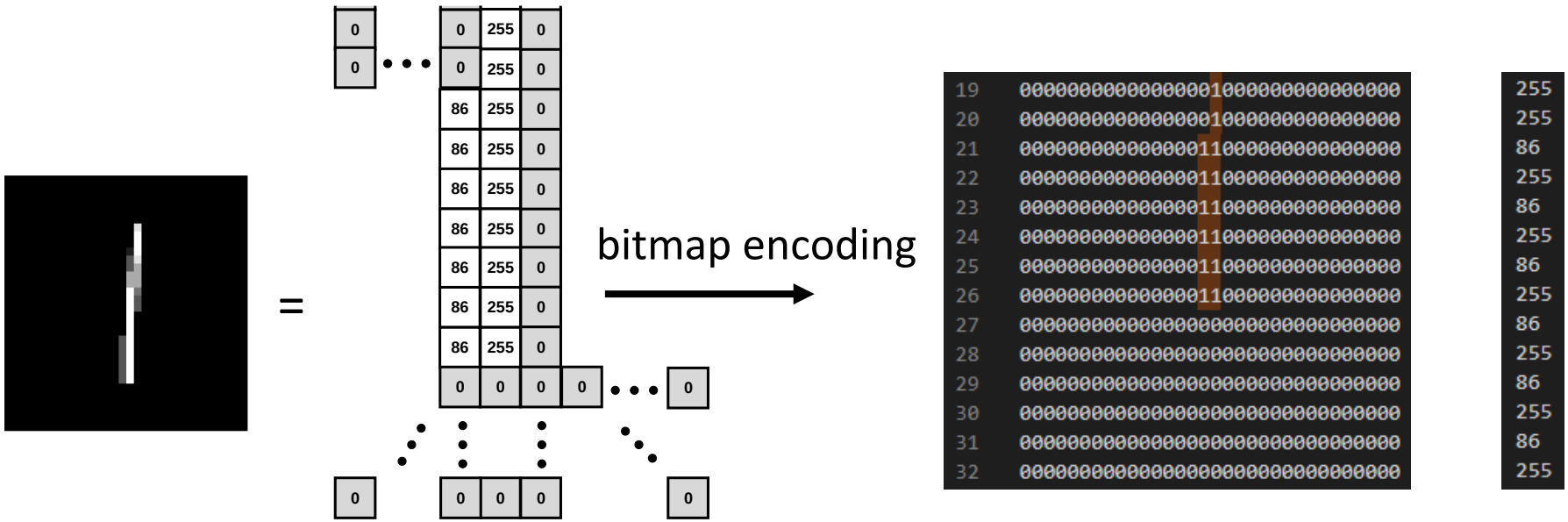
Weight Stationery



Source: MIT

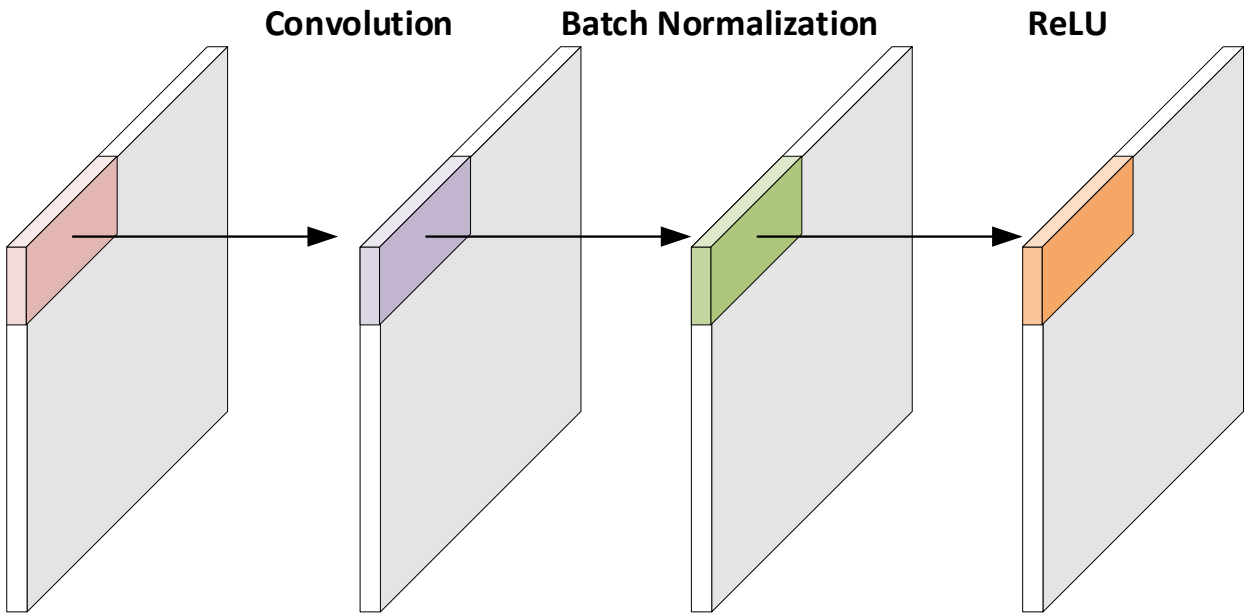
Model Encoding

- Model encoding can reduce the memory size of weight and computation requirement
- Common encoding method
  - Bit-mask encoding
  - Run-length encoding



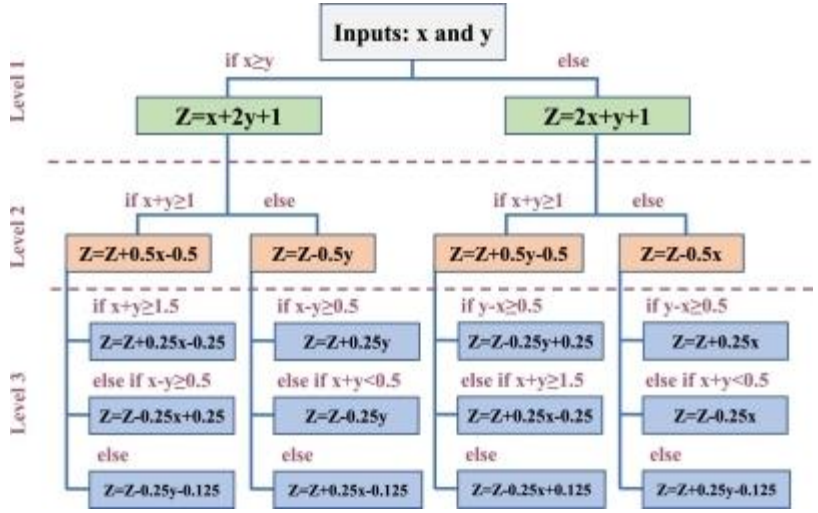
Kernel Fusion

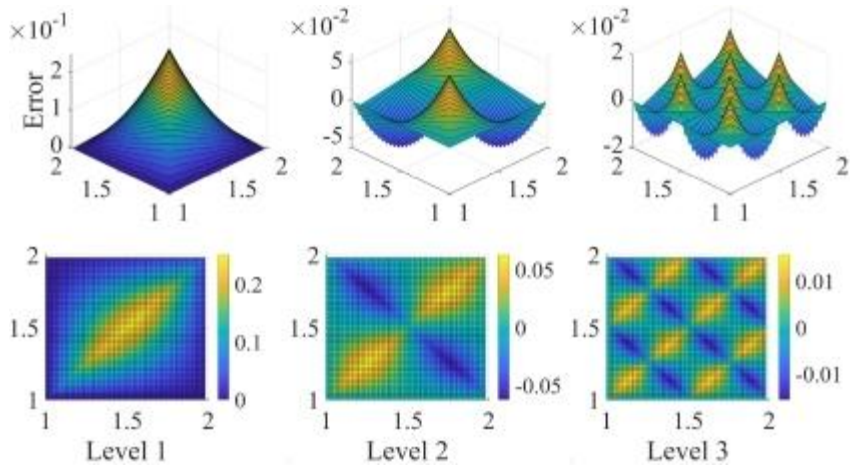
- The model may not need to be computed layer by layer
- Some operations can be fused to reuse the data and prevent massive data transfer



Approximate Computing

- Operate in ML does not need to be precise
- Some approximate computing technologies are proposed to reduce the latency/area of operators
  - For example, you may proximate the multiplication by addition





Source: [3]

Reference

- [1] Anaconda Installation: Download Anaconda Distribution | Anaconda
- [2] Run the Jupyter Notebook script: How to Safely Execute a Python Script in Jupyter Notebook
- [3] Approximate Multiplier: A Survey on Approximate Multiplier Designs for Energy Efficiency: From Algorithms to Circuits | ACM Transactions on Design Automation of Electronic Systems
- [4] Data Scheduling: eems.mit.edu/wp-content/uploads/2019/06/Tutorial-on-DNN-05-DNN-Accelerator-Architectures.pdf